

Active Exploration in SLAM: a Deep Reinforcement Learning Approach

Kai Wang¹, Teodor Ilie², Abdeali Harianawala², Paulo Araujo¹

¹Department of Electrical and Computer Engineering, Queen's University, Kingston, Canada

²School of Computing, Queen's University, Kingston, Canada

{k.wang, paulo.araujo, teodor.ilie, abdeali.h}@queensu.ca

Abstract—Active simultaneous localization and mapping (SLAM) aims to automate the traditional SLAM process where active selections of actions are not assumed. The core components of active SLAM are active error minimization and exploration. While standalone SLAM tools in the past decade have effectively solved the state estimation problem, the exploration has remained an open challenge. In this report, we propose a DRL-based active exploration framework that can be integrated in a general active SLAM pipeline. We design a pipeline with our DRL networks that leverages a prebuilt SLAM module and learns an optimal exploration policy in a self-supervised manner. We propose an efficient reward formulation based on our system characteristics for fast training and return convergence. Two RL algorithms: proximal policy optimization (PPO) and soft actor-critic (SAC) are compared. We tested our approach in a cubical, enclosed environment with occupied area in the center, in simulation. The result shows that the agent is able to train more than 0.5 times faster with our reward design compared to the raw reward. We also release our code open-source.¹

Index Terms—Active SLAM, Exploration, Deep Reinforcement Learning

I. INTRODUCTION

Simultaneous localization and mapping (SLAM) has been one of the most popular research topics in robotics. Often in the real world, a robot does not have prior knowledge about its environment, nor about its position within the environment. SLAM tackles these two challenges at the same time. In SLAM, the environment is represented as a map in different forms, such as occupancy grid, topological graph and geometric features [1]. During SLAM, the robot drives to explore the environment, while reducing the error in the estimation of its own pose and the locations of the features. The exploration task aims to maximize the robot's knowledge about the map. The reduction of the estimation error is typically realized by loop closure, i.e., the step in the robot path that allows it to revisit the past features. Traditional SLAM frameworks requires either a human operator or a preprogrammed path. Active SLAM, on the other hand, aims to automate such a process, i.e., to enable the robot to find an optimal path for exploration and error minimization [2].

In the past decades, there has been algorithms and frameworks developed for active exploration [3] and active loop closure [4]. With the growth of deep reinforcement learning (DRL) approaches in the recent years, robot agents have

been trained to explore both structured and unstructured environments [5]–[7]. The problem of active SLAM is often formulated as a partially observable markov decision process (POMDP) because of the following characteristics of active SLAM problems: (i) the agent makes decision about its path by learning from interaction with the environment, (ii) the states of the robot may not be fully observable, and (iii) the policies have to generalize to other scenarios without prior knowledge [2]. This gives rise to a class of learning-based approaches in solving active SLAM.

Thanks to contributions from the community of state estimation and computer vision, a great number of off-the-shelf passive SLAM frameworks have well solved the error minimization problem in robot pose and feature locations. These frameworks use different sensors as their main perception tools, e.g., ORB-SLAM3 [8] using monocular, stereo and RGB-D cameras and LOAM [9] using LiDARs. In [5], the authors used a passive SLAM module SE2LIO [10] to solve robot and feature localization, and a DRL module to tackle the exploration task. In this project, we follow this practice and focus on studying how DRL can be used to solve exploration in active SLAM.

This report is structured as follows: Section II introduces our problem formulation, DRL environment setup, state space, action space and general reward formulation. Section III covers the details of tool selection, learning algorithms and reward design. Section IV covers the project development process, including the difficulties, malfunctions and challenges encountered. Section V explains the training results and analysis. Section VI lists the open problems yet to be solved in this project. Our contributions in this project are: (i) An active SLAM pipeline that leverages a prebuilt SLAM module for filtered state estimation and accurate occupancy grid map building. (ii) We designed an efficient reward formulation for fast training. (iii) We compared the results of two classic RL algorithms on our environment and task formulation.

II. PROBLEM STATEMENT

In this project, we study active exploration problem in a SLAM task using DRL, which aims to maximize the knowledge of the robot about its environment. The knowledge is quantified using information gain. 2D occupancy grid is selected to represent the environment, as its information gain is the most straightforward to compute compared to other

¹https://github.com/kw908/cisc856_slam_rl

types of maps [11]. The agent used in this project is a lightweight, skid-steered ground mobile robot, controlled using commanded linear and angular velocities. As mentioned in Section I, the active SLAM problem is typically formulated as a POMDP. The action space is defined continuously as $[v, \omega]$ where v and ω are the linear and angular velocity of the robot. The range is $[(-v_{\max}, -\omega_{\max}), (v_{\max}, \omega_{\max})]$ where $v_{\max}$ and $\omega_{\max}$ is the absolute maximum linear velocity of the robot. The state space is the total number of cells explored together with the estimated 2D pose of the robot

$$S = \{\hat{x}, \hat{y}, \hat{\theta}, p(x_0), p(x_1), \dots, p(x_N)\} \quad (1)$$

The reward design is based on the information gain which is the change in the total number of explored cells, as detailed in Section III.

III. METHODOLOGY

A. System Design and Flow

An overview of our proposed system design is shown in Figure 2. The blue boxes represent the SLAM front-end, which takes sensor data from the 2D LiDAR, performs feature extraction and matching, and passes feature data and odometry measurements to the SLAM back-end, which then calculates pose graph optimization. Concurrently, as shown in the green boxes, sensor readings are used to construct an occupancy grid, which is transformed with Shannon entropy, $H_t(X)$, into an information gain reading for time step t , notated as ΔI_t :

$$H_t(X) = - \sum_{i=1}^n p_t(x_i) \log p_t(x_i) \quad (2a)$$

$$\Delta I_t = H_{t-1}(X) - H_t(X) \quad (2b)$$

Our initial reward function is a linear combination of these values, with additional penalties for collision and rewards for completing the majority of the map. This is passed to the RL algorithm to incentivize the robot both to perform a full map exploration and execute SLAM, while also rewarding more accurate pose estimation:

$$R_{\text{total}} = \lambda_1 R_{\Delta I_t} + \lambda_2 R_{\text{pose accuracy}} \quad (3)$$

Our system flow is shown in Figure 1; raw odometry and LiDAR data is fed into a SLAM module, which outputs both a pose estimate as well as an occupancy grid. These are combined in a reward function which is passed into a Deep Reinforcement Learning (DRL) model, outputting two actions - linear and angular velocity.

B. Tools and Platforms

Our approach was to simulate the environment in Gazebo², implement SLAM using the SLAM Toolbox³ and implement the RL component with PIC4rl [12]; all of these are connected with ROS2 as the middle-ware.

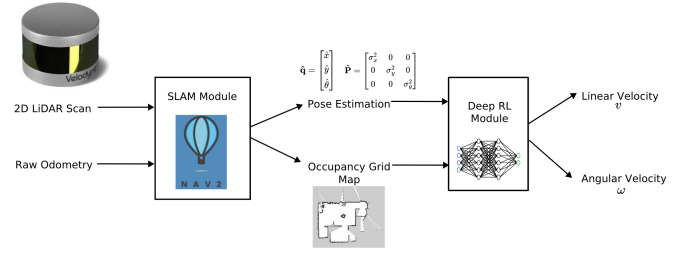


Fig. 1. End-to-end system flow diagram of our approach, demonstrating how we take raw odometry and LiDAR scan inputs, and output linear and angular velocities.

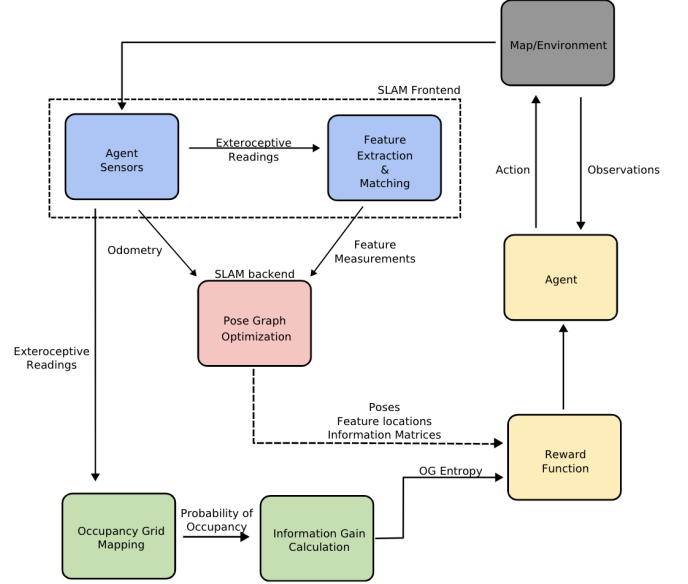


Fig. 2. Box diagram representing our system design. Components are colour-coded: SLAM front-end in blue, SLAM back-end in red, occupancy grid and info gain in green, and RL in yellow. The environment and map are in grey.

We chose to simulate our problem in a Gazebo environment to best integrate with the other components. Initially, we attempted to implement the simulation in simpler environments such as IR-SIM⁴ and Python Robotics⁵, as their lightweight nature might have simplified setup and sped up model training, but we found these did not implement all the necessary tools we intended to use. Once we decided to use Gazebo, and therefore ROS2, we chose to use the SLAM Toolbox, as it came with all the tools we required out of the box, and integrated seamlessly with ROS2/Gazebo. Similarly, PIC4rl supported our interfaces, and enabled us to implement a variety of relevant DRL algorithms. The world we constructed for training and testing is shown in Figure 3. By constructing a small world with a block in the centre, we wanted to see if our solution would enable the robot to learn how to navigate in a loop going around the block to see the entire accessible map.

²<https://gazebo.org/home>

³https://github.com/SteveMacenski/slam_toolbox

⁴<https://github.com/hanruihua/ir-sim>

⁵<https://github.com/AtsushiSakai/PythonRobotics>

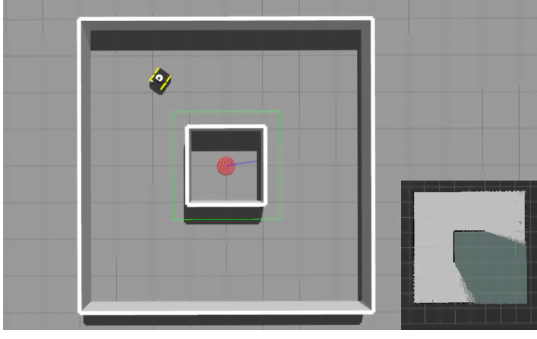


Fig. 3. Gazebo world visualization with the Jackal robot, with SLAM occupancy grid output in RViz in the lower right corner

C. Reward construction

Our initial reward function underwent several iterations as we progressed. Firstly, we discovered that the occupancy grid values coming from the SLAM Toolbox only held three discrete values - 0 and 100 for fully known spaces, and -1 for fully unknown spaces. As a result, it was possible to simplify the information gain formula into a simple difference of total known cells in the occupancy grid from time step to subsequent time step. This also implies that we no longer require prior knowledge of the map size to calculate information gain, only the restriction that the size remains fixed. The justification for this is shown below:

Shannon entropy is defined as in (2a). Note that the inner function can only take 2 values:

$$p_t(x_i) \log_2 p_t(x_i) = \begin{cases} 1, & \text{if } p_t(x_i) = 0.5 \\ 0, & \text{if } p_t(x_i) = 0 \text{ or } 1 \end{cases} \quad (4)$$

Let K_t be the total known cells at time t , U_t be the total unknown cells at time t , assuming a constant map size $U_{t-1} = U_t$, then

$$H_t(X) = \sum \mathbb{I}_{\{p_t(x_i)=0.5\}} \quad (5a)$$

$$= U_t - K_t \quad (5b)$$

$$\Delta I_t = H_{t-1}(X) - H_t(X) \quad (5c)$$

$$\Delta I_t = [U_{t-1} - K_{t-1}] - [U_t - K_t] \quad (5d)$$

$$\Delta I_t = K_t - K_{t-1} \quad (5e)$$

Additionally, during our testing, we found that the pose estimation done by the SLAM Toolbox was already very accurate, and we chose to remove this from our reward function. We then added a collision penalty reward to incentivize the robot to explore without colliding with environment objects, and added a large reward for more than 97% map coverage. Exploring this percentage of the map also served as our termination condition for an episode, in addition to termination in the event of a collision or of exceeding a pre-defined maximum time steps. These values were inspired by the similar work of Chen et al [5].

Furthermore, we noticed our information gain would be very high when the robot first spawned into the world, but then

subsequent gains would quickly diminish, and insufficiently motivate the robot to complete map exploration. This was an issue particular to our test environment, shown in Figure 3, as the robot would get a lot of information gain sooner after spawning, but then the gains received by completing exploration would be relatively low. For a different environment, such as exploration of a tunnel, this would not have been an issue, as information gain would be more or less constant as the robot continues to move forward.

To address this, we applied a logarithmic function to the information gain, such that large values are compressed and small values are magnified. This ensures that relatively small information gains in later stages of exploration still contribute meaningful rewards to the learning process and are not completely outweighed by the large information gain values in the initial steps of exploration. A comparison of these reward implementations is discussed in Section V.

Our final reward design is as follows:

$$R_t = R_{\text{cov}} + R_{\text{col}} + R_{\Delta I_t} \quad (6a)$$

$$R_{\text{cov}} = \begin{cases} 100, & \text{if map coverage} > 97\% \\ 0, & \text{otherwise} \end{cases} \quad (6b)$$

$$R_{\text{col}} = \begin{cases} -20, & \text{if collision} \\ 0, & \text{otherwise} \end{cases} \quad (6c)$$

$$R_{\Delta I_t} = \alpha \log(\beta \Delta I_t + 1) \quad (6d)$$

where $\Delta I_t = \text{curr_known} - \text{prev_known}$.

IV. DEVELOPMENT AND CHALLENGES

During our development, we faced several challenges, which we categorized as Environment Setup, Training Speed, RL Algorithm Selection and Other Errors. These are covered below. For an overview of the reward function shaping please see Section III-C above.

A. Environment Setup

Setting up our environment was significantly more challenging and time-consuming than anticipated. While the initial setup of the appropriate version of ROS2 and Gazebo was fairly straightforward, subsequent integration with the SLAM Toolbox and PIC4rl caused several issues that took longer to diagnose and fix than expected. Once this was complete, it took us some time to familiarize ourselves with the PIC4rl codebase, and debugging was slowed because we were unable to use a debugger while ROS2 and Gazebo were running. The PIC4rl codebase itself also had several errors that required debugging and fixing.

Following the initial setup, we also had to understand how to build a world in Gazebo and set a default viewpoint for easy visualization. When using some of the default worlds we noticed odd behaviours, including invisible objects, so we opted to create our own environment as shown in Figure 3, which was designed to train in a reasonable time. Next, we also had to configure the range of the LiDAR, as the default range was too far and would map most of the world, even if it

was a fairly large world, from the first moment, limiting our ability to perform exploration in a small map.

B. Training Speed

Upon completion of the basic environment setup, another challenge was speeding up the training. Due to the large computational overhead of the Gazebo simulation and the SLAM Toolbox, we attempted various strategies to speed the simulation time, which included running it without the GUI, increasing the speed, and enabling GPU use. With these, we encountered several issues.

Firstly, we found disabling the GUI minimally impacted training speed, as most of the computation appeared to be happening in the back-end, so we decided to run it with GUI even in training to observe the robot’s behaviour. We spent many hours attempting other strategies for speeding training, as each episode was taking anywhere from 10 to 30 seconds. To this end, we attempted training at higher speeds using the built-in “real_time_factor”, but this caused the robot to receive insufficient feedback from the RL algorithm output, and so it was unable to steer in time to avoid collisions. We also attempted to enable GPU support for several hours but eventually decided to abandon this, as most of the processing was not image-based, and we reasoned this would not speed up training significantly, and was not worth further time investment.

In the end, we had to accept the relatively slow training speed, which limited our ability to train as fast and often as we would have liked. Because training took around 10 hours for approximately 6000 – 8000 time steps - the minimum required for convergence - we opted instead to focus on our algorithm selection and reward function construction to speed the learning through that instead.

C. RL Algorithm Selection

In order to speed up training and achieve convergence, we carefully considered our algorithm choice. As we were using PIC4rl, which came with pre-defined RL algorithms out-of-box, we opted to choose from one of these. As we had already decided on a continuous action space, we initially attempted PPO as a reasonable standard on which to build a baseline model. We chose it as it is a stable and reliable policy-gradient method that requires little hyperparameter tuning.

After our first few runs, as describe in Section V, we decided to try a different algorithm that would converge faster. Choosing from our other options, we debated between SAC and TD3 and chose SAC. We opted for SAC as it explicitly encourages exploration, a key requirement for Active SLAM tasks. SAC, or Soft-Actor-Critic optimizes both the expected reward and the entropy, uses off policy learning that makes it more sample efficient, and employs entropy tuning to modify the level of exploration throughout the learning process. While TD3 - Twin Delayed DDPG - may have worked as well, it is more focused on accuracy, not exploration, which may not have been as ideal for our problem. Additionally, it may require more hyperparameter tuning, which we did not have

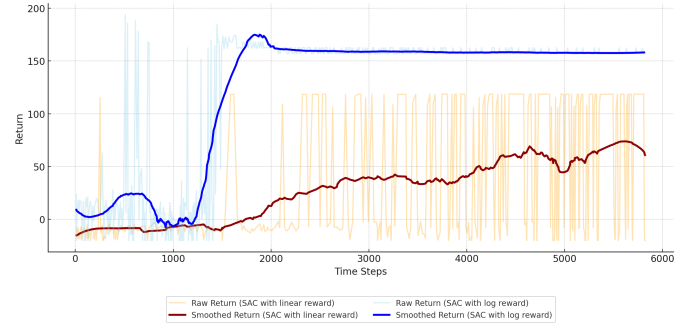


Fig. 4. SAC comparison with initial linear reward and modified log reward function. It can be clearly seen that the log reward sped up convergence. With log reward, the policy converged in around 2000 time steps, whereas with a linear reward, it had still not converged after 8000.

the time for, given the length of time required for convergence during training.

D. Other Errors

A few other notable errors occurred during development. In one of our runs, we found the robot had flipped over an invisible object, which was one reason we opted to build a custom map that we had full knowledge of. In another, the robot somehow disappeared from the map entirely, and we were unable to determine the cause. Finally, we discovered it was particularly challenging to reset the map generated by the SLAM Toolbox. This was necessary at the start of each episode to generate a meaningful information gain value. We attempted to use the existing topics for the serialization and deserialization of maps to load an empty map at the start of each episode, but after many errors, we scrapped this. Instead, we opted for killing and restarting the SLAM process itself. This method works almost all of the time but does cause occasional errors - given more time, we would like to address this more rigorously.

V. RESULTS

In this section, we describe results obtained during our training process using two reinforcement learning algorithms: Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC). The goal was to train an agent to autonomously explore and build a map of the environment using a reward function based on information gain in the occupancy grid, while penalizing collisions. We managed to get four training runs with meaningful results considering the time limitations we had. Here are the following modifications and observations made during experimentation:

A. Training Run 1: PPO with Default Max Steps Per Episode

In the first run, we used PPO from the PIC4rl library without modifying the default maximum number of steps per episode. During training in Gazebo, we observed that the robot was flipped over and remained stationary for the rest of the episode. Since the environment was not reset until the maximum number of time steps were completed, this resulted

in most of the training data being collected while the robot was stationary and flipped. As a result, the PPO model did not learn any useful policy, and the episode returns were consistently low and flat.

B. Training Run 2: PPO with Scaled Rewards and Early Termination

To improve training, we modified the maximum number of steps per episode to reset the simulation when the robot flipped or was stuck and we also decreased the maximum linear velocity of the robot so that it won't flip when crashes too fast into the wall. Additionally, we adjusted the reward function to scale the collision penalty appropriately. Initially, the collision penalty was set to a large negative value (-1000), which was disproportionately high compared to the positive reward from information gain. This imbalance led the agent to prioritize avoiding collisions over exploring the map. After scaling the collision penalty relative to the information gain, the reward function provided more meaningful feedback. Although PPO is generally considered stable and relatively easy to tune, our implementation may have suffered from sample inefficiency and insufficient runtime. Despite several improvements to the reward function, PPO did not show signs of convergence. This suggests that additional tuning or longer training durations may be necessary for effective learning for PPO.

C. Training Run 3: SAC with Scaled Rewards.

After reviewing related literature and documentation, we switched to the SAC algorithm, which is more suitable for continuous state and action spaces. SAC is an off-policy algorithm that can reuse past experiences through a replay buffer and optimizes for both reward maximization and entropy, encouraging better exploration. With the same reward scaling and early termination strategy as the second run, SAC began to show promising results, as seen in Figure 4. The training curve indicated that the policy started converging much faster than PPO. This improvement can be attributed to SAC's better sample efficiency and ability to handle the continuous navigation task more effectively.

D. Training Run 4: SAC with Logarithmic Reward Scaling

In the final training run, we observed that the agent received a very large reward due to high information gain immediately after being spawned in an unexplored map. However, as the robot continued to explore and the map became more complete, the incremental information gain reduced significantly. This caused the reward signal in later stages of exploration to become relatively small and less meaningful, which hindered further learning and refinement of the policy.

To address this, we applied a **logarithmic scaling** to the information gain component of the reward function, as described in Section III-C. With this adjustment, the SAC model continued to converge and showed more consistent exploration behavior throughout the episode, not just in the initial steps. This modification led to a better distribution of exploration effort over time and improved the overall map

coverage performance of the agent. The comparison is shown Figure 4. A video demo of training and testing is available at <https://youtu.be/ueNf1zpZSVA>.

VI. CONCLUSION AND FUTURE WORKS

In this project, we studied how to solve exploration in an active SLAM framework using deep reinforcement learning. The SLAM Toolbox served as a pre-integrated module that provided us with fast and accurate estimation for the robot pose and the occupancy grid map. We were then able to develop our DRL module and to reshape our rewards. The logarithmic scaling on our information gain reward significantly improves the training speed. The SAC algorithm proves to be a feasible and effective choice for our current environment and task formulation.

While our project demonstrated the potential of reinforcement learning for Active SLAM and exploration, there are several directions we plan to explore in the future to improve the performance and robustness of our approach:

- **Larger and Diverse Simulation Environments:** Our current experiments were limited to a single environment. In the future, we aim to train and evaluate our models across multiple and larger simulation maps. This will provide the agent with more opportunities for exploration and will help us understand how well the learned policy generalizes to unseen areas.
- **Hyperparameter Tuning:** In this project, we used the default hyperparameters provided by the PIC4rl library. We did not experiment with different values for learning rate, batch size, or discount factors. In future work, we plan to perform systematic hyperparameter tuning to improve the stability and performance of both PPO and SAC models.
- **Faster Training in Simulation:** One of the major challenges we faced was the slow training speed in the Gazebo environment, where each time step could take 10–15 seconds. We will explore optimization techniques to speed up simulation, such as using lightweight environments or faster and stronger computational resources.
- **Improved Reward Function Design:** Although we experimented with different forms of reward functions, including scaling and logarithmic transformations, we believe there is still room for improvement. We plan to investigate more approaches to designing reward functions that better capture exploration efficiency and mapping quality.
- **Removing Map Size Assumption:** Our current setup assumes that the size of the map is known in advance, which influences how the reward is calculated. In future work, we aim to remove this assumption and train policies that can explore and map environments of unknown size without prior knowledge.
- **Real-World Testing:** So far, all experiments were conducted in a simulated environment using ROS 2 and Gazebo. To validate the applicability of our approach in real-world scenarios, we plan to test our trained

policies on physical robots in actual indoor or outdoor environments.

REFERENCES

- [1] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. Leonard, “Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age,” *IEEE Transactions on Robotics*, vol. 32, p. 1309–1332, 2016.
- [2] J. A. Placed, J. Strader, H. Carrillo, N. Atanasov, V. Indelman, L. Carlone, and J. A. Castellanos, “A survey on active simultaneous localization and mapping: State of the art and new frontiers,” *IEEE Transactions on Robotics*, vol. 39, p. 1686–1705, 2023.
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.
- [4] C. Stachniss, D. Hahnel, and W. Burgard, “Exploration with active loop-closing for fastslam,” in *Proceedings of 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 1505–1510, 2004.
- [5] J. Chen, K. Wu, M. Hu, P. N. Suganthan, and A. Makur, “LiDAR-Based End-to-End Active SLAM Using Deep Reinforcement Learning in Large-Scale Environments,” *IEEE Transactions on Vehicular Technology*, vol. 73, pp. 14187–14200, Oct. 2024. Conference Name: IEEE Transactions on Vehicular Technology.
- [6] F. Niroui, K. Zhang, Z. Kashino, and G. Nejat, “Deep Reinforcement Learning Robot for Search and Rescue Applications: Exploration in Unknown Cluttered Environments,” *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 610–617, 2019.
- [7] F. Niroui, K. Zhang, Z. Kashino, and G. Nejat, “Deep reinforcement learning robot for search and rescue applications: Exploration in unknown cluttered environments,” *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 610–617, 2019.
- [8] C. Campos, R. Elvira, J. J. Gomez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] J. Zhang and S. Singh, “LOAM: Lidar Odometry and Mapping in Real-time,” in *Robotics: Science and Systems (RSS)*, 2014.
- [10] J. Chen, H. Wang, M. Hu, and P. N. Suganthan, “Versatile LiDAR-inertial odometry with SE(2) constraints for ground vehicles,” *IEEE Robotics and Automation Letters*, vol. 8, pp. 3486–3493, June 2023.
- [11] S. Thrun, “Exploration and model building in mobile robot domains,” in *Proceedings of IEEE International Conference on Neural Networks*, vol. 1, pp. 175–180, 1993.
- [12] M. Martini, A. Eirale, S. Cerrato, and M. Chiaberge, “PIC4rl-gym: a ROS2 modular framework for Robots Autonomous Navigation with Deep Reinforcement Learning,” Nov. 2022. arXiv:2211.10714 [cs].